When you finish this chapter, you'll have a solid understanding of the DI concept. In chapter 9 you'll see how to apply DI to your own classes. You'll learn how to configure your app so that the ASP.NET Core framework can create your classes for you, removing the pain of having to create new objects in your code manually. You'll learn how to control how long your objects are used and some of the pitfalls to be aware of when you write your own applications. In chapter 31 we'll look at some advanced ways to use DI, including how to wire up a third-party DI container.

For now, though, let's get back to basics. What is DI, and why should you care about it?

8.1 Understanding the benefits of dependency injection

This section aims to give you a basic understanding of what DI is and why you should care about it. The topic itself extends far beyond the reach of this single chapter. If you want a deeper background, I highly recommend checking out Martin Fowler's articles online. This article from 2004, for example, is a classic: http://mng.bz/pPJ8.

TIP For a more directly applicable read with many examples in C#, I recommend picking up *Dependency Injection Principles, Practices, and Patterns*, by Steven van Deursen and Mark Seemann (Manning, 2019).

The ASP.NET Core framework has been designed from the ground up to be modular and to adhere to good software engineering practices. As with anything in software, what is considered to be best practice varies over time, but for object-oriented programming, the SOLID principles have held up well.